

Indeksy i wydajność realizacji zapytań SQL

Indexes and the performance of SQL queries

dr hab. inż. Maciej Grzenda

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska

Wydział Matematyki i Nauk Informatycznych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Challenges related to querying and modifying data

- Does it matter whether a frequently executed query runs for 0.1 seconds or 5 minutes?
 - Will such a difference affect the user who is waiting for the query results?
 - Could it also have an indirect impact on other users of the system?
- If so, is there a way to reduce the execution time of SQL queries—both SELECT statements and those that modify data?
- Can we analyse how SQL statements are executed in order to optimise and minimise their runtime?
- To address these and other challenges, indexes should be utilised, and query execution plans should be carefully analysed.
- Indexes greatly affect how long queries take to run.

Objectives

- A DBMS platform is a core component of many core enterprise software systems
- While the business logic code, such as the algorithms calculating discounts for individual orders, can be executed within hundreds of milliseconds, SQL statements can take as much as several minutes or even hours to complete
- Hence, it is extremely important to apply techniques reducing the time needed to execute SQL statements
- These techniques are largely based on eliminating the need to scan entire table content to execute SQL statement with particular emphasis on SQL SELECT queries. To eliminate such a need, indexes (indeksy) are used

Tables in relational database – data storage

- Records in a table can be stored:
 - unsorted
 - or sorted i.e. physically saved on a disk is a certain sequence based on the value(s) of table column(s)
- By sorting records before writing them on a disk, data access time can be improved
- However, frequently there is a need to sort and search the records using different conditions (e.g. by customer name, address etc.)

Index (Indeks)

- Index is an on-disk structure associated with a table or view. It is used to speed up retrieval of rows from the table or view.
- Typically:
 - B-trees are used as an index representation
 - an index refers to the data from a single table rather than a view
- A core feature of an index is an index key (*klucz indeksowy*). In its base form, it takes the form of:
`Column1 [ASC|DESC], Column2 [ASC|DESC], ...`
- Sample index key definition:
`Country, City, Street, Address`
- An index will speed up queries e.g. including in its WHERE a condition `ColumnName=IndexColumn1` e.g. `WHERE Country='Belgium'`
- Designing indexes is a part of developing a physical data model

Clustered indexes

Indeksy zgrupowane

- To define a clustered index on a table means to make the DBMS physically store the records of the table in a specified order e.g. of growing values of OrderId column
- There can be at most one clustered index per a table, because the entire table i.e. table records are stored in this order
- Typically the records are stored in a B-tree, the leaves of the tree contain data pages
- When a clustered index exists for a table, the table is referred to as a clustered table (*tabela zgrupowana*)

The allocation of table data on a disk: clustered index – an example

- Example: [Order Details] table with a clustered index on OrderId
- The records in the table are physically stored in the order of OrderId values

OrderId	ProductId	Quantity	
1890			
1891			
1891			
1893			

The records are physically placed on a disk in the sequence of growing OrderId values. This may mean an extra overhead when the data is modified/inserted to preserve this sequence.

The allocation of table data on a disk: heap (sterta)

- If there is no clustered index created for a table, the records are stored in the form of a heap
- There is no specified order for storing the records of such a table
- Thus, records can be possibly inserted and updated faster as there is no need to preserve any order when inserting and updating records

Creating indexes: CREATE INDEX statement

- CREATE [UNIQUE] [CLUSTERED | NONCLUSTERED] INDEX index_name ON <object> (column [ASC | DESC] [,...n])
- Examples:
 - CREATE INDEX i1 ON Orders (CustomerId)
 - CREATE INDEX i2 ON Orders (ShipCountry,ShipCity)
 - CREATE CLUSTERED INDEX i3 ON [Order Details] (OrderId)
- By default a NONCLUSTERED index is created
- Note that only core settings of Ms SQL Server syntax are shown

Further details on CREATE INDEX statement on Ms SQL Server 2019 can be found at <https://docs.microsoft.com/en-us/sql/t-sql/statements/create-index-transact-sql?view=sql-server-ver15>

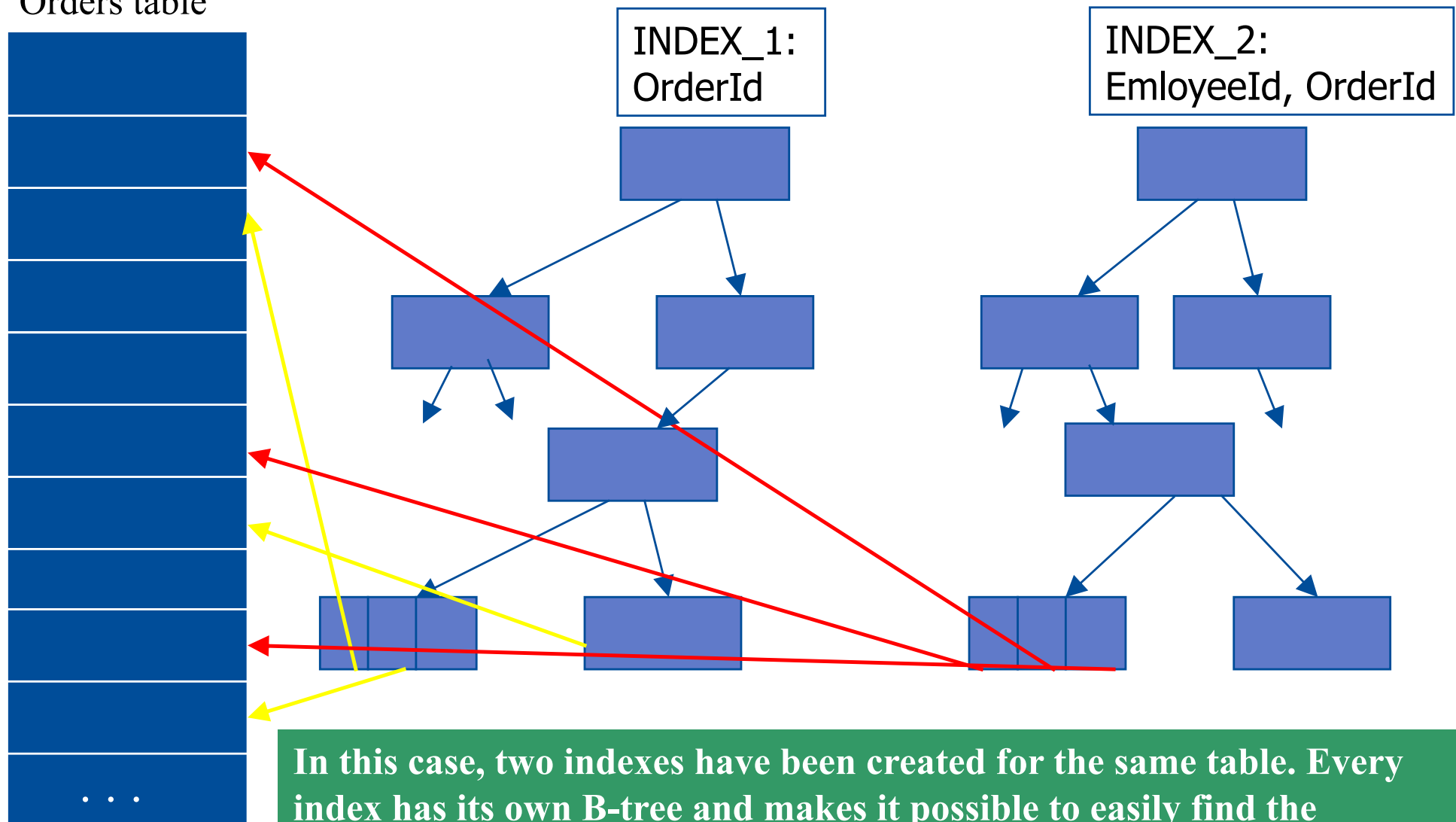
Nonclustered indexes

Indeksy niezgrupowane

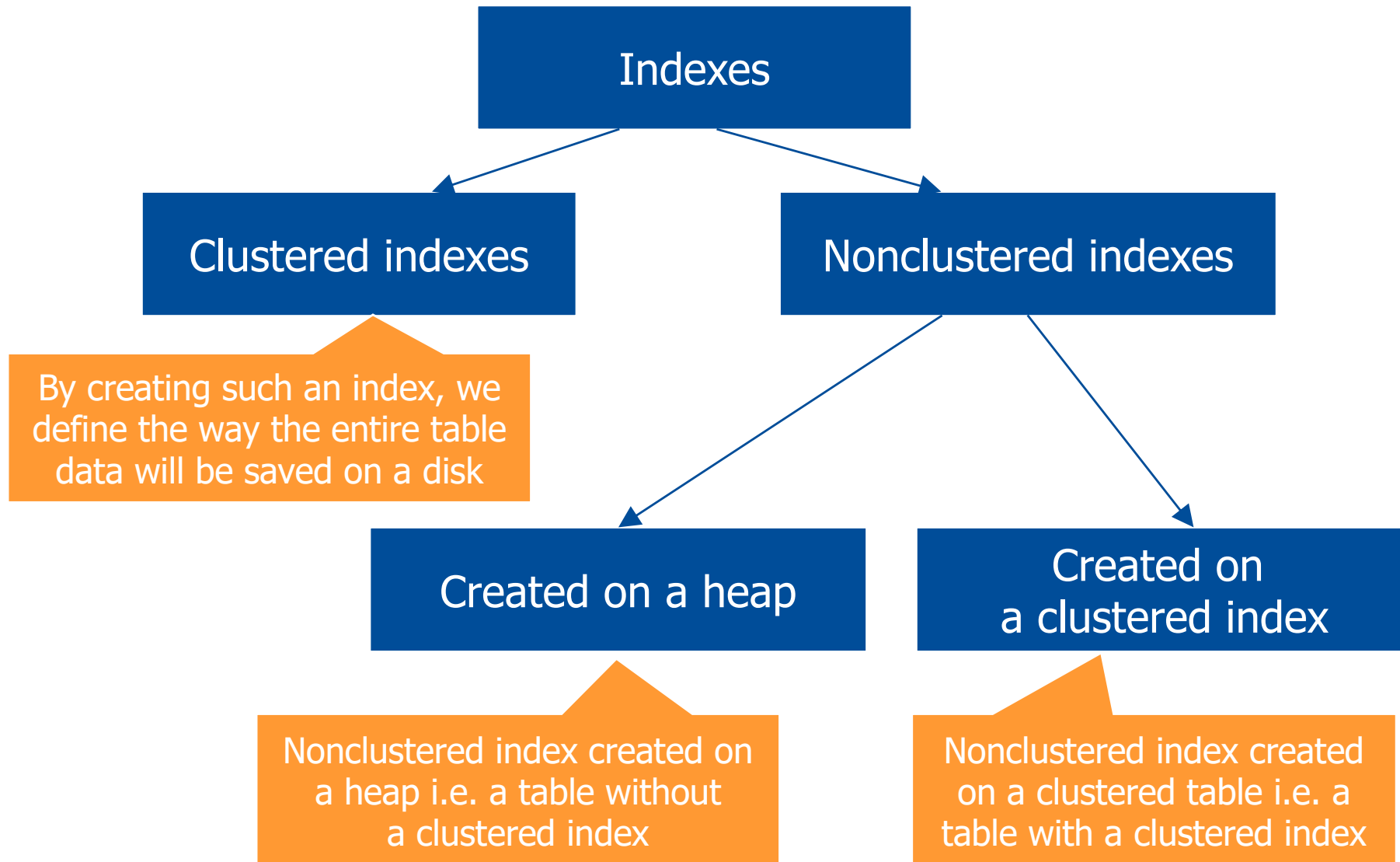
- There can be many nonclustered indexes for one table
- Every nonclustered index has a separate structure and requires its own extra space allocation on a disc
- For instance for one table, three nonclustered indexes with the following index keys can be created:
 - INDEX_1: Country, City, Street, Address
 - INDEX_2: CustomerName
 - INDEX_3: CustomerID

Nonclustered indexes – sample indexes for Orders table

The records of
Orders table



Index categories (Ms SQL Server)



Nonclustered indexes

- An index tree contains only the values of the index key i.e. the values of columns comprising on the index key
- Thus, the tree contains pointers to the records of the table
- Still, the index tree data i.e. index key values can be sufficient to answer queries, which need the values of index key columns only.
- Ms SQL Server makes it possible to add non-key columns to the leaf level of the nonclustered index to make it possible to answer more queries in this way

Indexes - objectives

- Key objective of creating an index is to improve performance while searching for records matching a condition (WHERE and JOIN clause) or sorting them (ORDER BY)
- Examples:
 - Index in [Order Details] on OrderId can be used to easily find all the order details records belonging to order 1045
 - Index on Country, City, Street, Address can be used to sort the customers by Country, City, Street, Address

Negative impact of indexes

- Once the index is created, it becomes the DBMS responsibility to keep the index tree up to date i.e. consistent with the table content
- Indexes can increase the time needed to complete INSERT/UPDATE/DELETE (CRUD) statements
- CRUD commands may require many index trees to be rebuilt. It is mandatory to update affected indexes.

Unique indexes *Indeksy unikalne*

- Can be created by a DBMS to preserve uniqueness of an index key (usu. a primary key of the table)
- Example: unique index is created for CustomerId column in Customers table
- Why?
 - The DBMS while updating the index can easily determine if there is another record with the same value of an index key
 - Without using an index the DBMS would have to search the whole table for a conflicting value

Indexes - guidelines

- The true advantage of an index is due to decreased number of disk blocks that must be read to execute an SQL statement
- Instead of scanning the whole table content and reading possibly megabytes of data, a DBMS can read significantly lower number of bytes defining an index structure and some of the records from the table
- Consider defining clustered indexes on a table whenever you frequently access group of related records e.g. multiple lines of an order. Then all the required records will be stored in the same part of the disk and read together.

Indexes – when to use?

- In each case:
 - To preserve the uniqueness of a primary key
 - To speed up referential integrity checking i.e. indexes should be created on foreign keys
- When large number of records is expected in a table (>1000) and:
 - the column(s) is/are frequently used in search/sort operations
 - The size of an index key is short (not to create huge index trees)
 - There is a good selectivity of the index column(s)

Referential integrity and indexes - example

Rooms

Building	Floor	Room
Main Building	2	213
...	..	...



Assets

Building	Asset	Room
Main Building	Chair	213
Main Building	Table	213



1. An index should be created on the columns of a primary key
2. An index should be created on the columns of a foreign key, otherwise every time a record in the rooms table is removed (or key value is modified) all the records in Assets will have to be scanned. Similarly every JOIN will need scanning the child table.

Indexes – when to avoid?

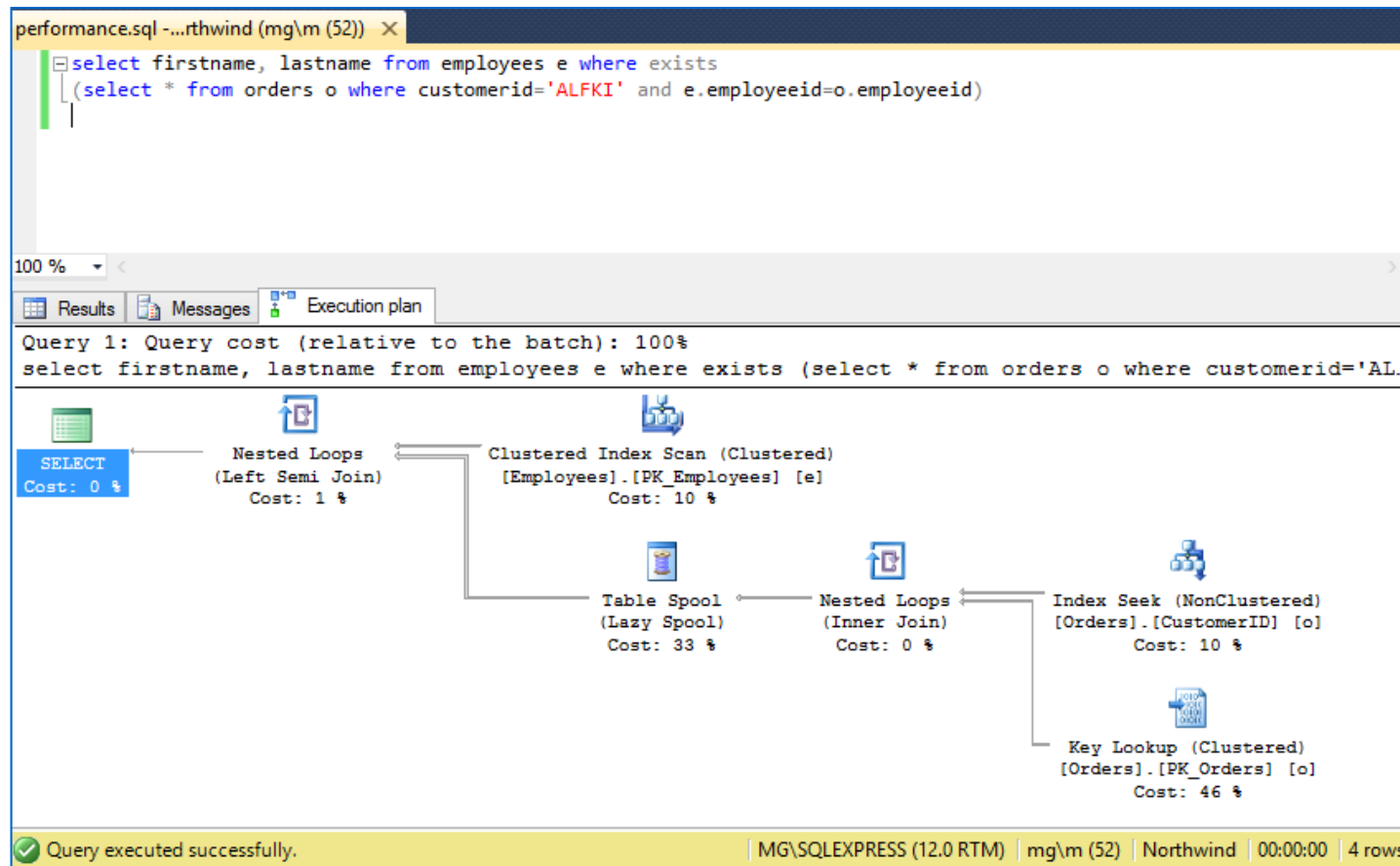
- Indexes increase processing time of INSERT/UPDATE/DELETE statements, thus avoid creating unnecessary indexes
- Example:
 - A column study_type takes only two values 'Bachelor of Science' and 'Master Of Science' (very poor selectivity), thus:
 - on average it can help filter out only half of the records,
 - still both index tree and half of the table content need to be searched, which can even increase processing time
 - As a result DBMS is not likely to use this index tree, but will have to update it.

Query execution plan

Plan wykonania zapytania

- DBMS to answer an SQL statement has to:
 - Develop query execution plan being an algorithm of executing the statement
 - The plan may include a number of steps
 - Steps include scanning index trees and table content
 - Whenever large tables have to be scanned, as there is no suitable index to be used, unacceptable response time can be expected
- Query execution plans can be investigated to identify and eliminate bottlenecks in data processing

Query execution plan – an example



It is possible to investigate query execution plans. On some DBMS like on Oracle RDBMS a request can be made to search for a better plan and/or propose new indexes that would speed up query execution by making better plans feasible.

Automated indexes: Oracle DBMS

In Oracle 19c edition released in 2019, automated indexing was introduced. This feature, introduced in Enterprise Edition:

- identifies possible indexes based on column use
- creates invisible indexes to consider their use when executing SQL statements
- converts invisible indexes found to improve the performance of SQL queries into visible ones. Unsuccessful invisible indexes are removed.

Note that this approach similarly to other approaches of this kind relies on statistical analysis of real SQL statements to find indexes possibly improving the performance of future SQL statements. Sudden changes in data use may require extra time to adapt indexes.

Further details: <https://oracle-base.com/articles/19c/automatic-indexing-19c>

Automated tuning: Ms SQL Server

In Ms SQL Server 2017, automated tuning was introduced. This feature, which is now present in Azure SQL Database (i.e. Database as a Service (DBaaS) based on Ms SQL Server):

- identifies problematic query execution plans and fixes them e.g. by restoring previous better plans when plan choice regression is detected
- Identifies indexes that should be added and indexes that should be removed
- Enables automatic tuning in Azure SQL database

Note that this approach similarly to other approaches of this kind relies on learning the characteristics of SQL workload to search for possible improvements.

Further details: <https://docs.microsoft.com/en-us/sql/relational-databases/automatic-tuning/automatic-tuning?view=sql-server-ver15>

Summary

- Relational databases require indexes to offer acceptable performance
- Hence, indexes are a mandatory part of any relational database deployment
- In fact, the ability to create multiple indexes per a table is a major advantage of relational systems, usually unavailable in Big Data platforms
- Query execution plans can be investigated to reveal the reasons of unsatisfactory query execution, usually meaning too long execution time
- Leading DBMS vendors have been developing automated indexing approaches for many years
- Still, a properly planned database should include indexes matching expected workload even before this workload happens

Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html





**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.